

CSC477 – INTRODUCTION TO MOBILE ROBOTICS

ASSIGNMENT 1, 15 POINTS

DUE: OCTOBER 12, 2026, 6PM ET

Course Page: https://csc477.github.io/2026F_website/

Overview: In this assignment you will write a feedback controller that enables a ground robot, which obeys differential drive dynamics, to follow a wall. The purpose of this assignment is to make you develop experience with the following concepts:

- The Robot Operating System (ROS 2): its architecture and its publisher-subscriber model, which abstracts away the details of distributed computation and message passing from the robot programmer. Familiarity will also be developed with ROS 2's main visualization tool, called `rviz2`.
- Controlling the yaw of a ground robot via PID control.
- Processing 2D laser measurements.
- The Gazebo simulator, which is one of the most popular simulators in the robotics community.

The software stack is ROS 2 Humble, Gazebo Fortress and the Clearpath simulator, all on Ubuntu 22.04. The robot is a simulated Clearpath Husky A200 carrying a 2D laser scanner.

Get the starter code: The starter code is in a public git repository:

```
git clone https://github.com/csc477/assignment1.git
```

It contains the ROS 2 package `wall_following_assignment` (the code you edit and submit), a `docker/` directory with the recommended development environment, a few helper scripts in `scripts/`, and two setup guides in `docs/`. The `README.md` files in the repository repeat and expand on the instructions below; when in doubt, the repository is the more up-to-date of the two.

Setting up your development environment: You have two supported options. Use the first one if at all possible.

Option 1 (recommended): Docker on your own computer. The repository ships a Dockerfile that builds an image with exactly the software the lab machines have (Ubuntu 22.04, ROS 2 Humble, Gazebo Fortress, the Clearpath simulator, at pinned package versions). The container displays Gazebo and `rviz2` on your desktop, and the assignment package is mounted from your clone, so you edit with your usual editor on the host. You need:

- A Linux host with a graphical desktop, or Windows 11 with WSL2 running Ubuntu (WSLg provides the display). macOS is not supported: Gazebo's 3D view needs OpenGL 3.3, which XQuartz cannot provide to a container. Mac users should use Option 2.
- Docker Engine (<https://docs.docker.com/engine/install/>), working from your normal user account, i.e. `docker run hello-world` succeeds without `sudo`.
- About 8 GB of free disk. No GPU is needed; software rendering is fast enough for this assignment.

Then:

```
cd assignment1/docker
./run.sh build      # once; downloads about 5 GB
./run.sh shell      # a shell inside the container; repeat for more terminals
```

Inside the container your workspace is `~/csc477_ws`, and your clone of the repository appears inside it as `src/assignment1`, the same layout as on the lab machines. Build it:

```
cd ~/csc477_ws
colcon build --symlink-install
source install/setup.bash
```

See `docs/docker.md` in the repository for details and troubleshooting.

Option 2 (fallback): the MCS lab machines, remotely. If you cannot set up Docker (for example because you use a Mac), or the simulator is too slow on your computer, use one of the lab machines

`mn3110pc01.utm.utoronto.ca`, `mn3110pc02.utm.utoronto.ca`, ..., `mn3110pc16.utm.utoronto.ca`

They have everything preinstalled, a GPU for rendering, and a remote desktop server, so you can work from anywhere. Your home directory is network-mounted and identical on all of them, so it does not matter which one you pick and you can switch machines whenever the one you are on is busy. You log in with your UTORid and UTORid password. From off campus you need to use UTorVPN.

- **Remote desktop** (for Gazebo, rviz2 and any other window): connect with an RDP client (Windows: *Remote Desktop Connection*; macOS: *Windows App*; Linux: *remmina* or *xfreerdp*) to the machine's full hostname, accept the self-signed certificate, leave the session type as Xorg, and log in. You get your own desktop. Disconnecting keeps it running; reconnect later and everything is still there.
- **SSH** (for editing and terminals): `ssh utorid@mn3110pc01.utm.utoronto.ca`, or VS Code with the *Remote - SSH* extension.

One-time setup in your account on the lab machine (ROS 2 is already sourced in every shell there):

```
rosdep update
mkdir -p ~/csc477_ws/src && cd ~/csc477_ws/src
git clone https://github.com/csc477/assignment1.git
cd ~/csc477_ws && colcon build --symlink-install && source install/setup.bash
```

See `docs/remote_lab.md` in the repository for screen-size settings, isolation between students, and troubleshooting.

Option 3: native installation on Ubuntu 22.04. If you run Ubuntu 22.04 (Jammy) with sudo access, you can install ROS 2 Humble and the simulator directly. This is the fastest option, especially with a GPU. The trade-off is that your installation is not identical to ours, so the course staff may have a harder time helping with environment problems; if you get stuck, moving to Docker or the lab machines is always possible, since the workspace layout is the same. ROS 2 Humble only exists for Ubuntu 22.04; on 24.04 or later, use Docker. The full command list is in `docs/native.md` in the repository; in short, add the ROS 2 and Clearpath apt sources and install

```
sudo apt install ros-humble-desktop ros-dev-tools \
  ros-humble-clearpath-simulator ros-humble-clearpath-desktop \
  ros-humble-rqt-reconfigure ros-humble-teleop-twist-keyboard \
  ignition-fortress python3-pytest
```

then `sudo rosdep init`, `rosdep update`, `source /opt/ros/humble/setup.bash` from your `~/.bashrc`, and create the workspace exactly as in Option 2.

Whichever option you use, we expect your code to build and run in the Docker image, and that is where it will be graded.

Run the starter code: We have created a few simulated worlds consisting of sequences of walls, and a simulated ground robot, for the purposes of this assignment. Bring up a world with walls in the Gazebo simulator (this also spawns the robot):

```
ros2 launch wall_following_assignment gazebo_world.launch.py world:=walls_one_sided
```

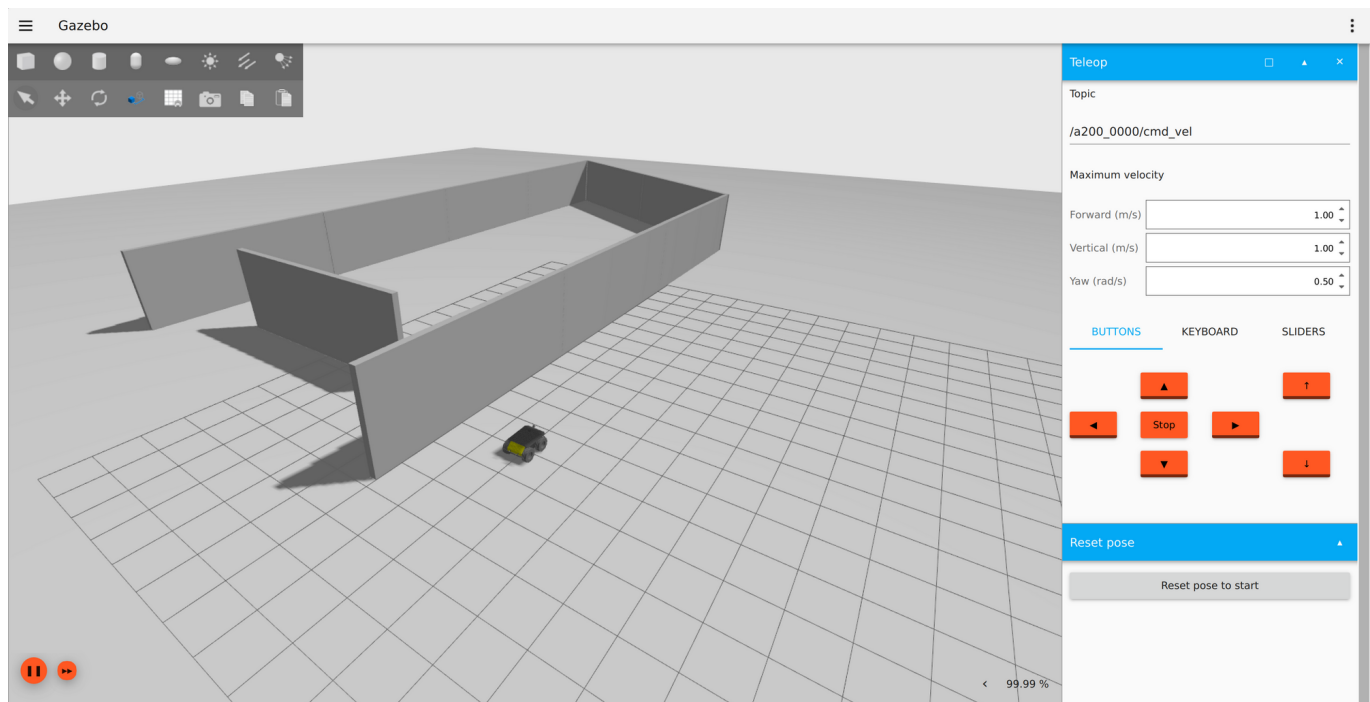


Figure 1: Gazebo environment with walls only on one side of the robot. The Teleop panel on the right lets you drive the robot with the mouse or keyboard, and the *Reset pose to start* button below it teleports the robot back to its starting pose.

The first launch takes a while. The Gazebo window in Fig. 1 should appear, with the Husky standing next to the first wall, on its left. If instead you get errors, please post them on the course discussion board. Then, in a second terminal, start rviz2, which is the default visualization system for ROS 2, with the course configuration:

```
PKG=$(ros2 pkg prefix wall_following_assignment)/share/wall_following_assignment
rviz2 -d $PKG/config/csc477.rviz --ros-args -p use_sim_time:=true \
  -r /tf:=/a200_0000/tf -r /tf_static:=/a200_0000/tf_static
```

The two remappings and the `use_sim_time` parameter are required: the robot publishes its transforms inside its namespace, and without them rviz2 cannot place the laser scan in the world. You should see what is shown in Fig. 2.

The red line is a set of points detected by the simulated 2D laser. The axes represent the tree of reference frames that the system is aware of. If all of this works you are ready to proceed to the next section, which is the essence of the assignment. At this point you can run a simple last check: drive the robot by hand, either with the Teleop panel in the Gazebo window or from a terminal:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -r cmd_vel:=/a200_0000/cmd_vel
```

This brings up a node publishing on the topic `/a200_0000/cmd_vel` and prints the key bindings (i forward, j/l turn, k stop). If you can move the Husky around with the keyboard you are ready to write your PID controller. Should the robot end up stuck against a wall, click *Reset pose to start* in the Gazebo window.

Implement and test a wall-following controller (15 pts) The input to the robot will be laser scan messages from the robot's simulated laser scanner. See https://docs.ros2.org/latest/api/sensor_msgs/msg/LaserScan.html for the ROS 2 message definition. Also, make sure you have understood the ROS 2 tutorials about the publisher-subscriber model of distributed computing: <https://docs.ros.org/en/humble/Tutorials.html>. We have provided starter code for Python and C++ in the files

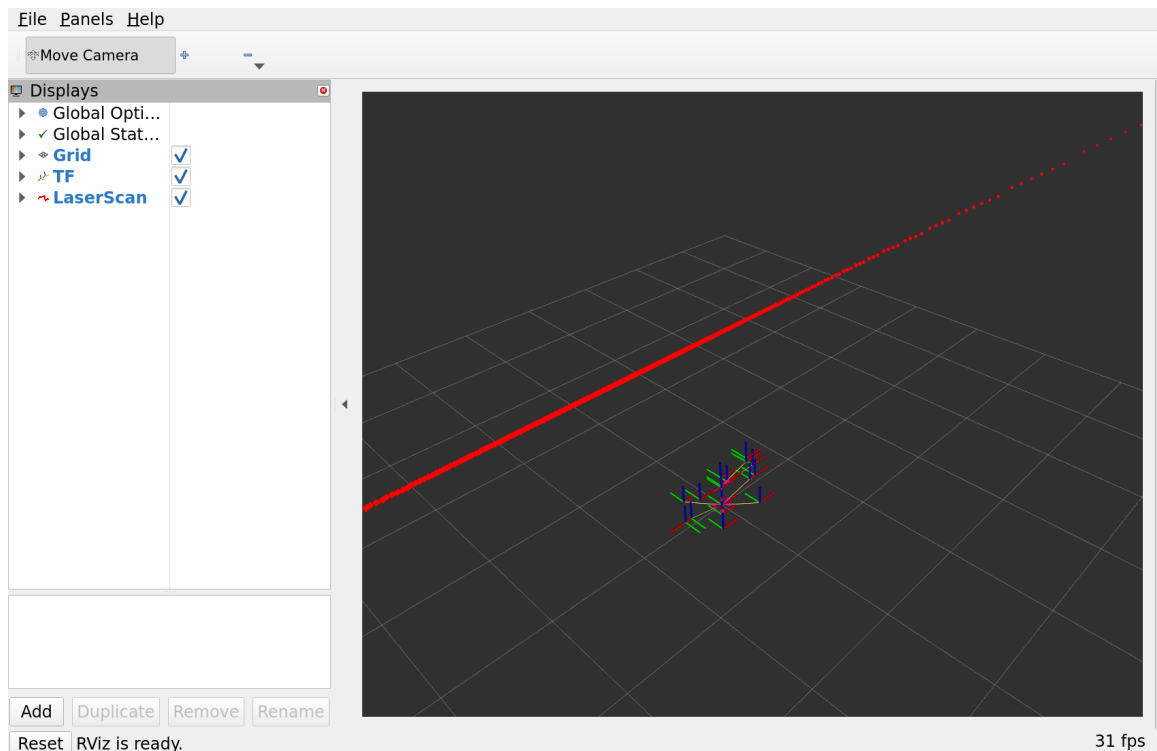


Figure 2: rviz2 visualization for the Husky robot in Fig. 1. The red points are the wall detected by the simulated 2D laser. The cluster of axes is the tree of reference frames from ROS 2's tf system.

wall_following_assignment/scripts/wall_follower.py

wall_following_assignment/src/wall_follower_node.cpp

wall_following_assignment/include/wall_following_assignment/pid.h

Choose your language and edit the appropriate files. The C++ files contain TODOs. The Python file already contains a minimal controller of the simplest kind, which you should read, run, and improve. The topics your node uses are, all under the robot namespace a200_0000:

/a200_0000/sensors/lidar2d.0/scan	sensor_msgs/msg/LaserScan	subscribe
/a200_0000/cmd_vel	geometry_msgs/msg/Twist	publish
/a200_0000/cte	std_msgs/msg/Float32	publish

Specifically:

[Part A, 2 pts] Compute the cross-track error based on each incoming laser scan and publish it under the topic name /a200_0000/cte with the ROS 2 message type `std_msgs/msg/Float32`. Watch out for NaN and inf values in the scan ranges. The ROS 2 tutorials on writing a publisher and subscriber are at

- Python: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>
- C++: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html>

[Part B, 7 pts] Populate the PID class based on the provided API. For each incoming laser scan issue an angular velocity command to the robot on the topic /a200_0000/cmd_vel, based on the output of the PID controller. You need to follow the wall on the LEFT of the robot, as it is placed in its initial configuration. The starter code offers two ways to compute the error: (1) from the distance to the closest wall only, or (2) from that distance and the orientation of the wall relative to the robot. They are not equally good; think

about what the first one does at the corners of the course, and what information it is missing.

[Part C, 2 pts] Tune your controller's parameters in real time. The starter code already exposes the PID gains K_p , T_d and T_i as ROS 2 parameters of the `/wall_follower` node, so they can be changed while the node runs. Use

```
ros2 run rqt_reconfigure rqt_reconfigure
```

or `ros2 param set /wall_follower Kp 0.3` to tune them manually, and make your final gains the defaults in your code. Add any further parameters your controller needs (for example a look-ahead distance) the same way. A set of parameters is considered good enough, and the run is considered successful, if the robot does not collide with the wall and completes at least 3/4 of a full circuit around the left wall.

[Part D, 4 pts] For each of the two simple wall worlds in `wall_following_assignment/worlds/`, namely `walls_one_sided` and `walls_two_sided`, do the following.

Launch the world as above, then launch your wall-following controller like this:

```
ros2 launch wall_following_assignment wall_follower_python.launch.py
or
ros2 launch wall_following_assignment wall_follower_cpp.launch.py
```

according to your language of choice.

- (1pt/world) Use a desktop recording program (OBS Studio, SimpleScreenRecorder, the built-in recorders of Windows, macOS and GNOME, or similar) to record a video of your robot while it is following the wall. Be sure to include any failure cases. If you work on a lab machine, record the remote desktop window on your own computer.
- (1pt/world) Record the cross-track error published by your node as follows:

```
ros2 bag record -o my_cte /a200_0000/cte
```

and after your robot's run is done, convert the recorded messages in the bag into a text file with the script provided in the repository:

```
python3 ~/csc477_ws/src/assignment1/scripts/bag_to_csv.py \
my_cte cross_track_error.csv
```

Clean restarts. Gazebo's bridge processes do not always die with `ros2 launch`, and an orphaned bridge re-attaches to the next simulation you start, delivering every laser scan twice. If `ros2 topic info /a200_0000/sensors/lidar2d_0/scan` ever reports two publishers, run `scripts/teardown.sh` from the repository until it reports no survivors, then relaunch.

[Optional Part E, 1 bonus pt] Implement and evaluate the PID self-tuning algorithm that was mentioned in class. Plot the cross-track error as a function of epochs (or iterations), where an epoch is a round of evaluation of a PID parameter setting on the simulator.

[Optional Part F, 2 bonus pts] Evaluate your wall-following ROS 2 node on one of the real racecars that we have available for this course at UTM, shown in Fig. 3. Note that these racecars, as well as a technician to support their operation, are only available at UTM. If you are interested in completing this bonus question, please reach out to our robotics lab technician at UTM, Greg Zetko (greg.zetko@utoronto.ca).

Submission Instructions Assignment submissions will be done on Quercus. You will submit a zip file containing the following:

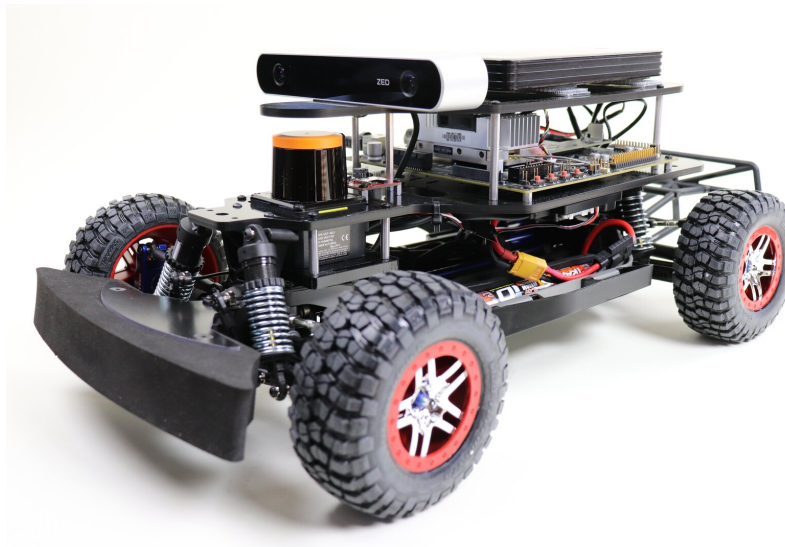


Figure 3: CSC477 Racecar

1. Your `wall_following_assignment` directory for Parts A, B, and C. Do not include `build/`, `install/` or `log/` directories, or bag files.
2. Two videos, one demonstrating the robot's navigation in the world `walls_one_sided` and another in the world `walls_two_sided`, as explained in Part D. The videos should be named as follows:

```
FirstName_LastName_StudentNumber_walls_one_sided.[mp4/avi]  
FirstName_LastName_StudentNumber_walls_two_sided.[mp4/avi]
```

Each video should not exceed 10MB.

3. Similarly, two csv files from Part D. They should be named:

```
FirstName_LastName_StudentNumber_walls_one_sided.csv  
FirstName_LastName_StudentNumber_walls_two_sided.csv
```

The point of including a video of your controller in your submission is not just for us to easily examine your code. It's also so that you can easily show your work later on to classmates/coworkers/employers. It becomes part of your portfolio. It is also worth noting that, due to the ROS abstraction layer, if you want, you could run your feedback controller on a real Husky robot without major modifications.

4. If you have done the optional bonus question E, please submit the resulting figure as:

```
FirstName_LastName_StudentNumber_bonus_question_PID_autotuning.[png/jpg]
```

5. If you have done the optional bonus question F, please submit a video (under 20MB) showing the racecar doing wall-following on the 3rd floor of Deerfield Hall at UTM:

```
FirstName_LastName_StudentNumber_bonus_question_Racecar.[mp4/avi]
```

We expect your code to build and run in the provided Docker image (the lab machines run the same software). If it does not you will lose marks. We expect to be able to compile it using `colcon build`. Once we compile your code we will run the following:

```
ros2 launch wall_following_assignment gazebo_world.launch.py world:=[world_name]
ros2 launch wall_following_assignment wall_follower_[language].launch.py
```

We will test your code on other worlds similar to the ones provided. We will also test it using different desired distances from the wall. The default desired distance is 1 meter, and the default forward speed is 1 m/s; both are set in the launch files, so do not hard-code them in your node. We will test your code with different parameters in that neighbourhood. Your code will also be examined for correctness, style and efficiency. We recommend that you come by during office hours or post on the discussion board if you are unsure of your implementation.